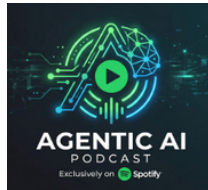


12 Module 9 — Centralized Multi-Agent Patterns

Code

Orchestrator-Worker, Hierarchy, Supervisor-Router, and Pipelines — the four ways a controller owns the control flow, and why three of them look alike until you draw them



Agentic AI Podcast — listen on Spotify

Companion podcast: [Al agents design new SARS-CoV-2 nanobodies](#) — a Principal-Investigator agent coordinates specialist agents to design real drugs — orchestrator-worker in the wild. · [browse all episodes](#)

12.1 Where this module sits

[Module 8](#) took the single agent as far as one agent goes: one model, one loop, one tool catalog, one growing context. It also gave you the honest list of reasons that design hits a ceiling — a constitution that grows past the rot zone, a tool catalog too large for the model to choose from cleanly, a reasoning trace that tangles high-level planning with low-level tool minutiae, and the plain fact that one agent runs sequentially. This module is the first step *off* that ceiling.

This is the **second Layer-2 module**. Recall the three-layer course model: Layer 1 (Modules 3–7) is the patterns *inside* one agent; Layer 2 (Modules 8–11) is how agents are *composed* into a system; Layer 3 (Modules 12–13) is how those compositions fail. Module 8 was the simplest Layer-2 composition — a system of one. Going multi-agent is the next escalation, and the escalation principle from Module 1 governs it exactly as it governed everything else: **you do not earn a second agent by wanting one. You earn it by naming the specific way the single agent failed.**

That justification matters more here than anywhere yet, because going multi-agent does not *remove* complexity — it **relocates** it. You trade an overstuffed single context for a coordination problem: agents that pass information back and forth, that can stall waiting on each other, that can disagree about who is in charge. The cost has a name we will use all module — the **coordination tax** — and every pattern below pays it. The honest reasons to pay it are a short list, and you should be able to recite the one that applies before you reach for any pattern here:

- **Context budget pressure** — one context is overflowing; splitting into agents with *isolated* contexts is often a context-engineering move in multi-agent clothing.
- **Conflicting prompts** — a great planner constitution, a great executor constitution, and a great critic constitution look nothing alike; merging them produces a system prompt that does none of the three jobs well.
- **Parallelism** — independent sub-tasks can run at once, cutting wall-clock latency.
- **Independent verification** — a second agent catches errors the first is biased against seeing in its own work.
- **Domain-specific tooling** — a retrieval sub-agent and a scoring sub-agent want different tools, and mixing the catalogs confuses both.

This module covers the **centralized** family: every pattern here has a single component that **owns the control flow** — a controller decides who runs, in what order, and when the work is done. That is the defining trait, and it is the lens for the whole module: centralized control trades flexibility for **predictability and debuggability**. (Module 10 gives away that control to peer agents, and Module 11 mixes the two.)

By the end of this module you should be able to:

1. State what makes a multi-agent pattern *centralized*, and price the coordination tax in the three currencies.
2. Draw the control flow of **Orchestrator-Worker**, **Supervisor-Router**, and **Sequential Pipeline** and explain why three patterns that look alike on a box-and-arrow sketch have genuinely different control-flow semantics.
3. Recognize **Hierarchy** as nested orchestration that should follow *problem* structure, and name the anti-pattern that follows when it follows org-chart instincts instead.
4. Price any pattern here in the three currencies and name each one's characteristic failure.

12.2 Centralized means a controller owns the control flow

Strip the vocabulary away and “centralized” answers one question: **when an agent finishes, who decides what happens next?** In a centralized pattern the answer is always the same — a *single, named component* decides. The workers, handlers, or pipeline stages do their piece and hand control *back up* (or *along*); they never decide among themselves who goes next. That single decider is the controller, and it is the thing every pattern in this module has and the patterns in Module 10 deliberately give up.

This is the same control-flow axis from Module 1, lifted from inside one agent to the whole system. There, the axis ran from *predetermined* (your code decides) to *model-driven* (the model decides). Here it runs from *centralized* (one controller decides) to *decentralized* (peers negotiate). The trade is the same shape it always is.

Watch — a controller dispatching workers in parallel. A short comparison: a single loop grinds through Retrieve → Score → Assemble sequentially, while a controller fans the same steps out to workers that never talk to each other, then aggregates the results.

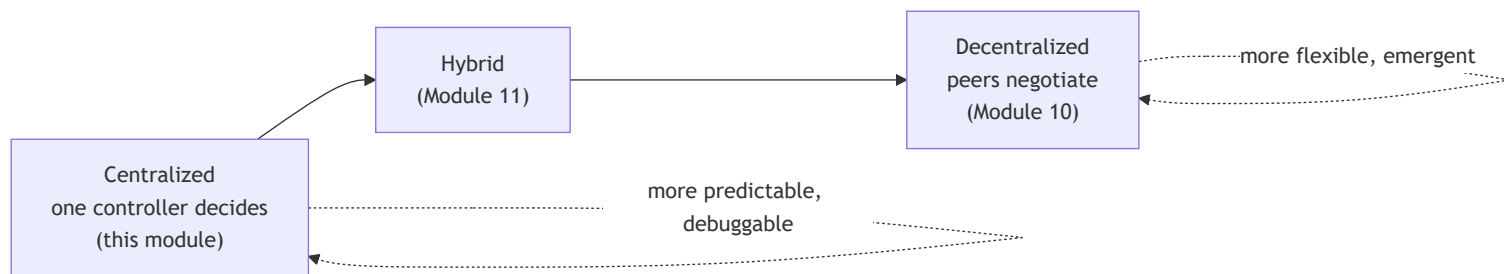
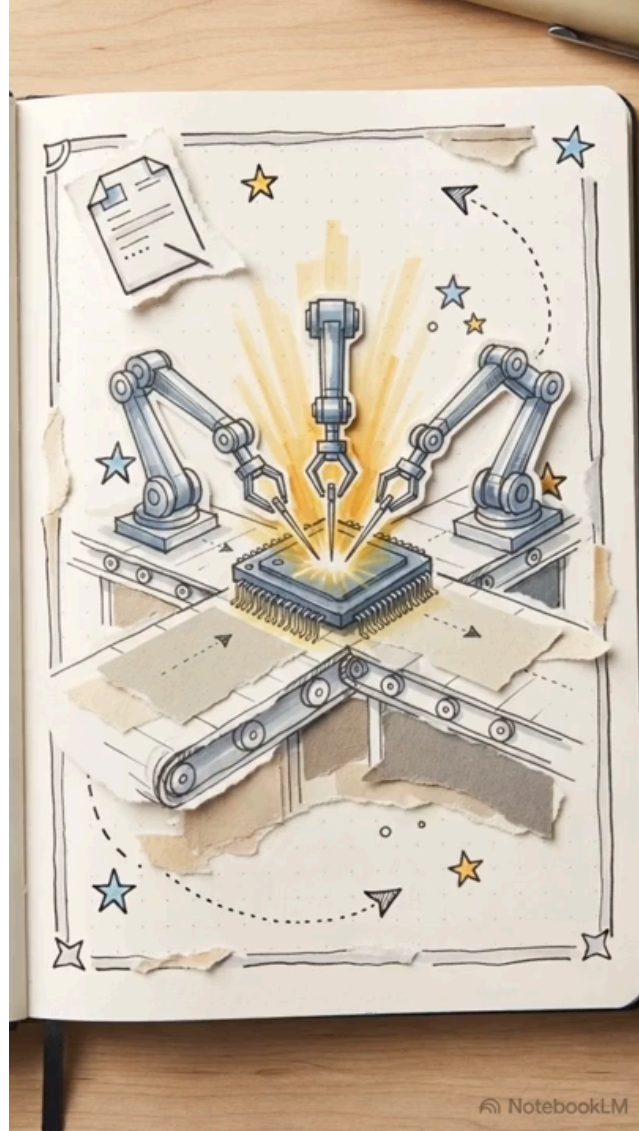


Figure 12.1: Centralized control on the system-level axis. A single controller owning the flow buys predictability and debuggability; pushing the decision out to peer agents (Module 10) buys flexibility at the cost of both. This is the Module 1 control-flow trade, lifted to the system level.

Why start centralized at all? Because it is the cheaper, safer point on the axis, and the escalation principle says start there. A controller you wrote is a controller you can read. When the system misbehaves, there is one place that decided the order, and you can inspect its reasoning. Decentralized systems trade that single inspection point for emergent flexibility — and, as Module 10 will show, for failure modes that are genuinely hard to debug because *no one component is in charge of the thing that went wrong*. Centralized first; decentralized only when the problem’s shape demands it.

12.3 Orchestrator-Worker — decompose, dispatch, aggregate

Orchestrator-Worker is the most common production multi-agent pattern, and it is common for a reason: it maps cleanly onto the most natural way to split work. One agent — the **orchestrator** — receives the task, *decomposes* it into self-contained sub-tasks, *dispatches* each to a **worker** agent, then *aggregates* the workers’ structured results into a final answer. The workers do not see each other; each gets a self-contained brief, does its piece, and returns a structured result up to the orchestrator.

Note the shape of the orchestrator’s job: it has a *beginning* (decompose), a *fan-out* (dispatch, often in parallel), and an *end* (aggregate). The workers are leaves; the orchestrator is the only place that holds the whole picture.

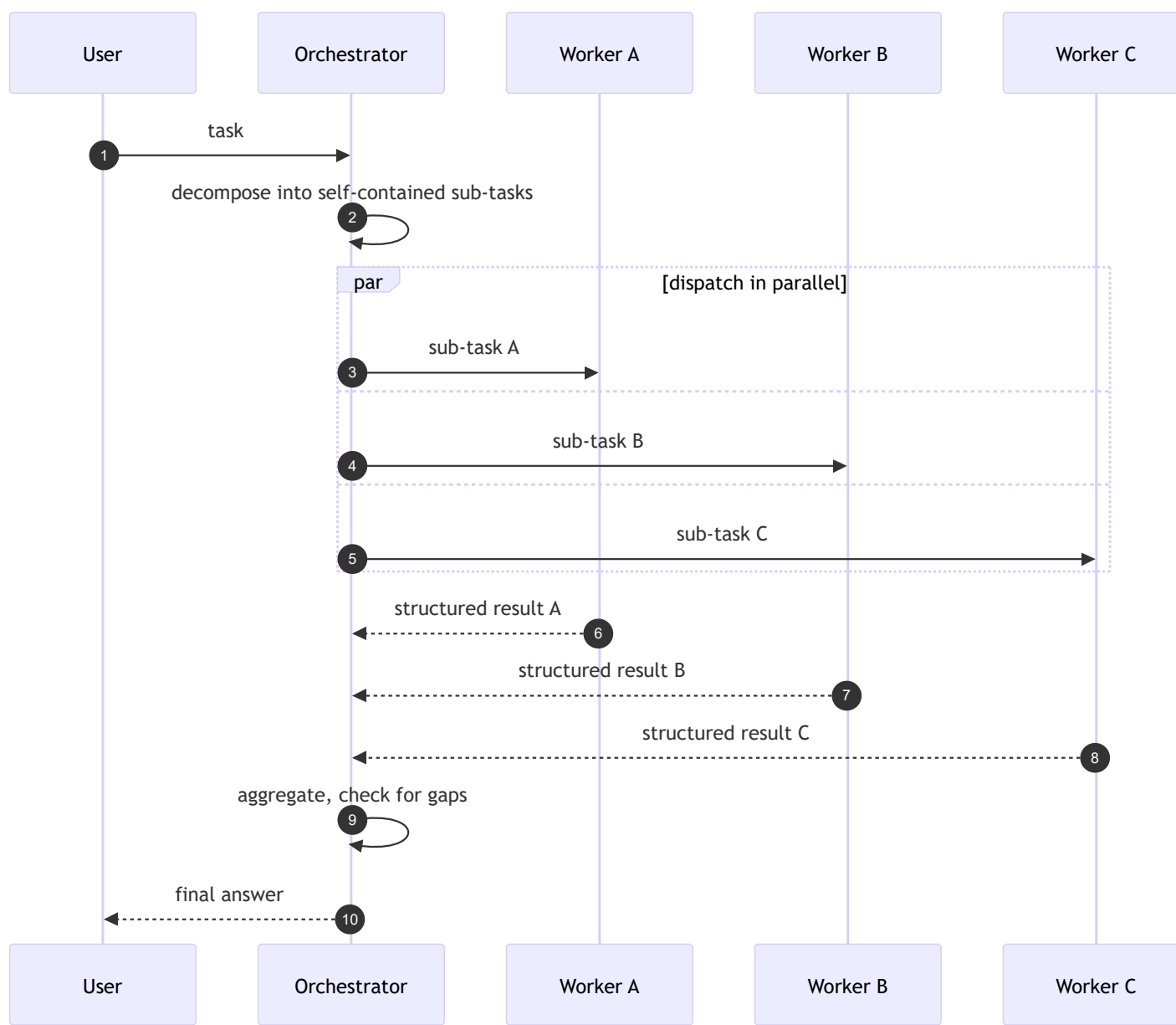


Figure 12.2: Orchestrator-Worker over time. The orchestrator decomposes once, fans the sub-tasks out to workers in parallel, blocks on their structured returns, then aggregates into a final answer. Control returns up to the orchestrator after every worker, never sideways between workers.

Read what the sequence diagram makes obvious. Every arrow from a worker goes **back up to the orchestrator** — never sideways to another worker. The orchestrator decomposes *once* at the top

and aggregates *once* at the bottom; the middle is a fan-out/fan-in. This is the control-flow signature of the pattern, and holding onto it is what will let you tell it apart from the look-alikes below.

The mid-term project is a clean example. “Build a team for this solicitation” decomposes naturally into **retrieve-candidates** (find faculty whose publications match the call), **score-fit** (rate each candidate against the solicitation), and **assemble** (pick a complementary team and write the justification). An orchestrator can dispatch retrieve-candidates and score-fit as workers — even fan scoring out one worker per candidate to run in parallel — then aggregate into the assembled team. Each worker has its own constitution and its own tools (the retrieval worker talks to the publication index; the scoring worker does not need to), which is the *conflicting-prompts* and *domain-tooling* reasons from the escalation list, both at once.

12.3.1 What Orchestrator-Worker costs

Currency	What Orchestrator-Worker spends
Latency	The decomposition call and the aggregation call are both on the critical path before and after the workers. But the workers themselves can run <i>in parallel</i> , so wall-clock latency can be <i>lower</i> than a single agent doing the same steps in series — this is parallelism’s whole payoff.
Tokens / money	More than a single agent for the same task — you pay for the orchestrator’s decompose and aggregate calls <i>on top of</i> all the worker calls. The coordination tax is most visible here.

Failure modes	The orchestrator can decompose badly — split into sub-tasks that miss part of the goal or
---------------	---

Currency	What Orchestrator-Worker spends
	overlap — and the aggregation step can paper over a worker that silently failed or returned a confident wrong result.

The characteristic failure is **bad decomposition**: the orchestrator carves the task into pieces that do not actually reconstitute the whole. A scoring worker handed a candidate the retrieval worker never found cannot score them; an aggregation step handed three partial answers can stitch together something that reads complete and is missing a quarter of the work. The deeper version of this failure is *over-decomposition* — splitting into far more, far smaller sub-tasks than the problem needs — which is severe enough to be a named architecture anti-pattern in **Module 12**, and which the Hierarchy section below will set you up to recognize.

12.4 Hierarchy — orchestrators of orchestrators

If a worker's sub-task is itself big enough to need decomposing, make the worker an orchestrator. That is the whole idea of **Hierarchy** (also called *Manager-of-Managers*): nested orchestration, where a top-level orchestrator dispatches to mid-level orchestrators, each of which dispatches to its own workers. It is Orchestrator-Worker applied recursively.

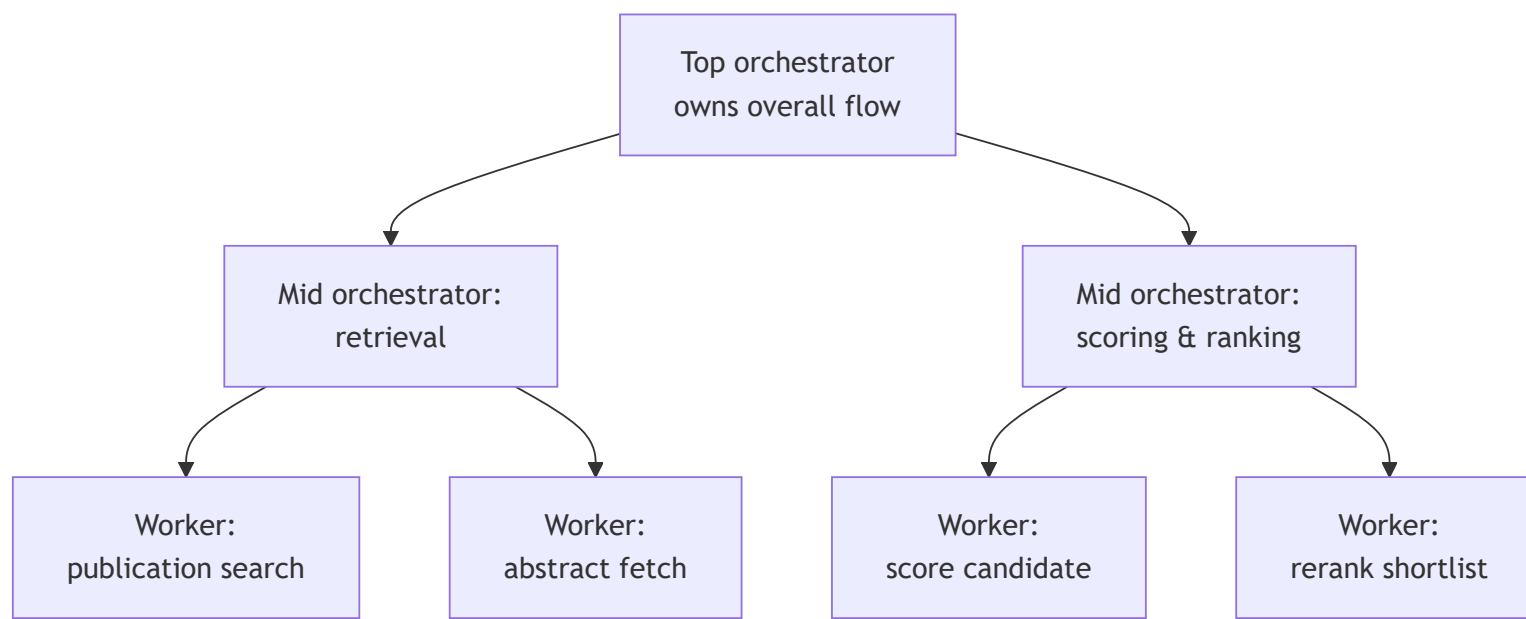


Figure 12.3: Hierarchy as nested orchestration. The top orchestrator owns the overall control flow and delegates whole sub-problems to mid-level orchestrators, each of which owns the control flow for its branch and delegates to leaf workers. Control flows strictly up and down the tree, never across branches.

The structure diagram shows the rule that keeps a hierarchy debuggable: control flows **up and down the tree, never across branches**. The scoring branch and the retrieval branch do not talk to each other; they each report to the top orchestrator, which is the only place the branches meet. Each node owns the control flow for its own subtree and nothing else. That is what preserves the “one place decided the order” property at every level — there is one such place per subtree.

Hierarchy earns its place when the problem is *genuinely* nested — when a sub-problem is large enough that decomposing it again is the honest description of the work. The trap is reaching for hierarchy because it *looks* organized. Here is the single most important judgment in this section:

Hierarchy should follow the structure of the problem, not the structure of an org chart.

Newcomers build hierarchies that mirror how a *human company* would staff the task — a “manager” agent, “senior” agents, “junior” agents — because that feels like good engineering. It is

not. Those layers add coordination tax (every level is another decompose-and-aggregate round-trip, more latency, more tokens, more places to fail) without adding any decomposition the problem actually called for. The org chart is a social structure for managing *people*; your problem has its own structure, and only *that* structure justifies a layer. If a sub-task fits in one agent’s context with one coherent constitution, it does not get its own orchestrator no matter how senior it sounds.

This is the on-ramp to **over-decomposition (Module 12)**: a hierarchy with more layers than the problem has natural seams is over-decomposition made structural. Every needless layer is pure coordination tax. When you draw a proposed hierarchy, ask of each layer: *what sub-problem is large enough that it had to be decomposed again here?* If you cannot answer for a layer, delete it.

12.5 Supervisor-Router — route the next turn

Here is where the module earns its keep. **Supervisor-Router** *looks* almost identical to Orchestrator-Worker on a box-and-arrow sketch — a central agent in the middle, several specialist agents around it, arrows both ways. It is the canonical LangGraph-style multi-agent shape ([primer 16](#)). But its control flow is genuinely different, and confusing the two is one of the most common mistakes in multi-agent design.

A **supervisor** does not decompose a task and fan it out. It looks at the *current conversation state* and decides **which single agent should handle the next turn**. That agent takes its turn, returns control to the supervisor, and the supervisor decides again — *given what just happened* — who goes next. It is a loop of routing decisions, one per turn, not a decompose-dispatch-aggregate.

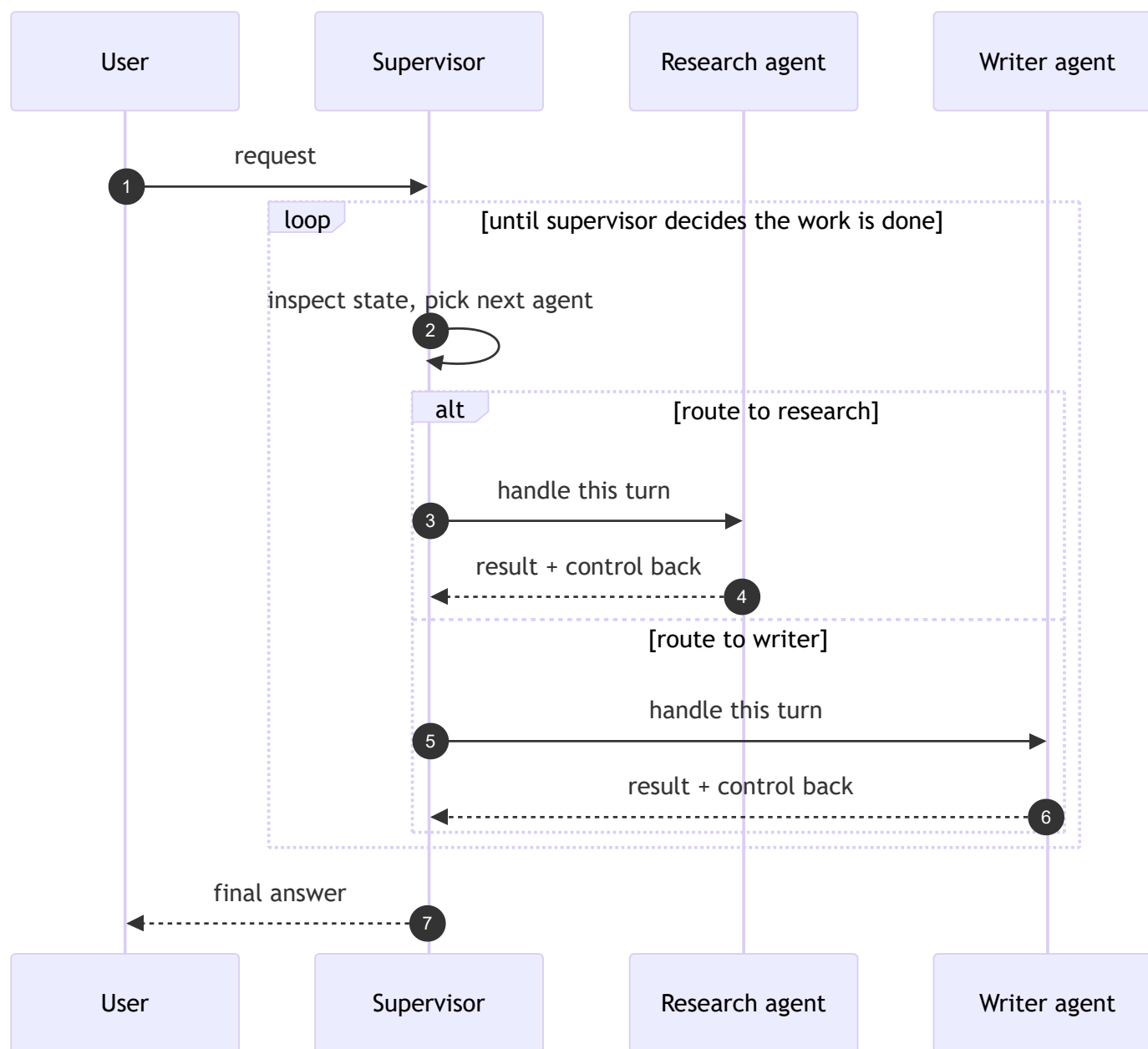


Figure 12.4: Supervisor-Router over time. The supervisor inspects the current state and routes a single turn to one specialist, which returns control to the supervisor. The supervisor then re-decides who goes next given what just happened. It is a per-turn routing loop, not a one-shot fan-out.

Now put the two sequence diagrams side by side and the difference is unmistakable. The orchestrator decided the *whole* plan up front and dispatched in parallel; the supervisor decides *one*

turn at a time and routes in series, re-deciding after each turn based on what just changed. The orchestrator is **Plan-and-Execute at the system level** — commit to a decomposition, then run it. The supervisor is **ReAct at the system level** — decide the next move from the latest observation, loop. (Those are the Module 3 reasoning patterns reappearing one layer up, which is exactly the “Layer 2 composes a reasoning loop inside it” claim from that module, now made literal: the *system’s* control flow has a ReAct or Plan-and-Execute shape of its own.)

That difference is not cosmetic — it changes how the system behaves:

	Orchestrator-Worker	Supervisor-Router
When the plan is decided	Once, up front (decompose)	Continuously, one turn at a time
Worker/agent execution	Often parallel fan-out	Strictly one agent per turn, in series
Adapts mid-task to new info	Only by re-decomposing	Natively — every routing decision sees the latest state
Control flow at the system level	Plan-and-Execute shaped	ReAct shaped
Predictability	Higher — read the decomposition	Lower — the route emerges turn by turn
Best when	The sub-tasks are knowable from the goal and independent	The right next specialist depends on what the last one produced

Because the supervisor decides every turn, it inherits ReAct’s failure mode at the system level: a routing loop with **no termination condition** runs forever — the *No Termination Criteria* anti-pattern (Module 13) reincarnated as a supervisor bouncing the conversation between two agents that keep handing it back. Every supervisor needs the same two exits a ReAct loop needs: the supervisor can declare the work done, *and* a hard cap (max turns, max tokens, wall-clock budget) can stop it regardless of what the supervisor wants.

12.5.1 What Supervisor-Router costs

Currency	What Supervisor-Router spends
Latency	A supervisor call <i>before every specialist turn</i> , all in series — there is no parallel fan-out to amortize it. Worst latency profile in this module for the same number of agent turns.
Tokens / money	The supervisor re-reads the conversation state on every routing decision, so its context grows turn by turn — the same context-re-send cost ReAct pays, now at the system level.
Failure modes	Misrouting (the supervisor sends the turn to the wrong specialist) and non-termination (the routing loop never decides it is done). The first is the system-level cousin of routing’s misclassification failure from Module 3.

The characteristic failure is **non-terminating ping-pong**: the supervisor routes to agent A, A returns asking for something only B can do, B returns asking for something only A can do, and the

supervisor faithfully shuttles between them forever. The defense is the hard cap — and, in the constitution, a clear definition of “done” the supervisor can recognize.

12.6 Sequential Pipeline — a fixed chain of transformations

The last pattern is the most predetermined of the four, and the cleanest. A **Sequential Pipeline** (or **Chain**) is a *fixed* sequence of agents, each transforming the previous one’s output and handing it to the next. Stage 1 → Stage 2 → Stage 3, in that order, always. There is no decomposition decision and no routing decision — the order is wired in by *you*, in code.

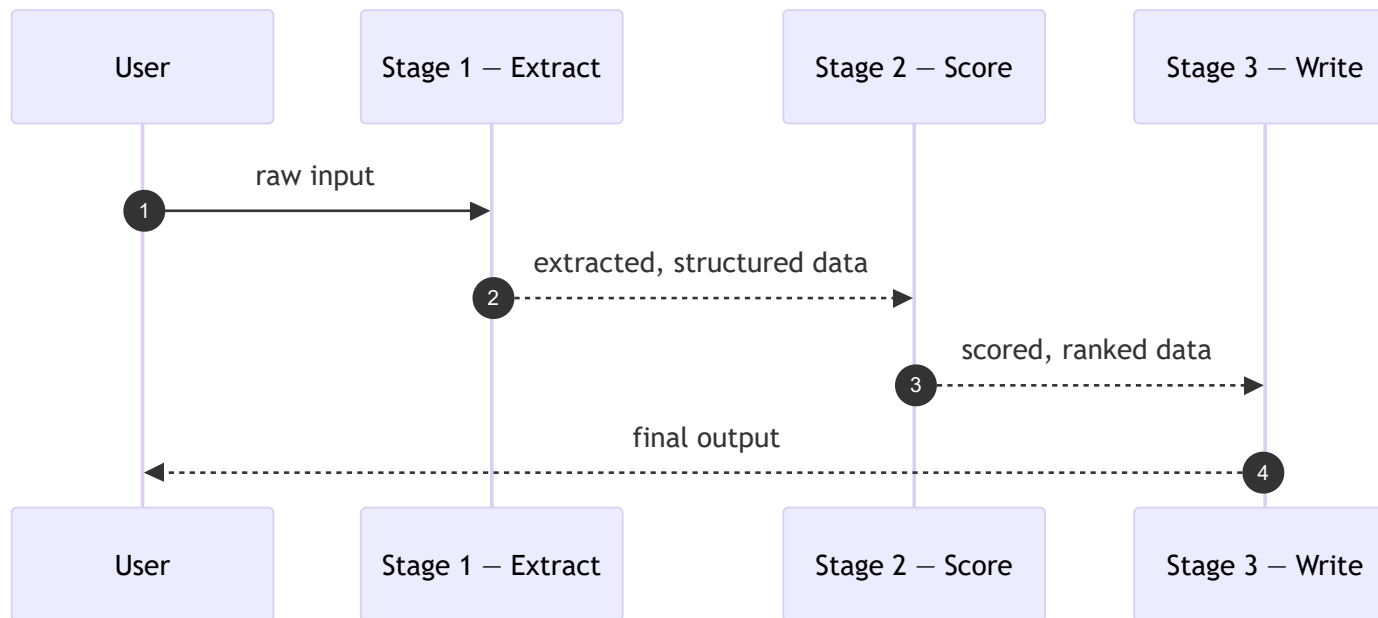


Figure 12.5: A Sequential Pipeline over time. Each stage transforms the previous stage’s output and hands it to the next, in a fixed order set by your code. No component decides the order at run time — the control flow is fully predetermined.

Look at what the pipeline’s sequence diagram *lacks*. There is no central controller in the middle of the picture taking control back between stages. Each stage hands its output directly to the next; the

“controller” is the static wiring you wrote, not a runtime agent making decisions. That makes a pipeline the most predictable, most debuggable, cheapest-to-reason-about pattern in the module — and it raises a question worth confronting head-on.

12.6.1 The workflow/agent boundary, revisited

Is a Sequential Pipeline even a multi-*agent* pattern, or is it a workflow with LLMs in the boxes? Recall the Module 1 definition: the line between workflow and agent is **who controls the flow** — your code (workflow) or the model (agent). By that definition a pure pipeline with a fixed order is a **workflow**, full stop. The order is predetermined; no model decides what happens next.

This is not a technicality — it is the most useful thing to notice about pipelines. **Pipelines routinely sit right on the workflow/agent boundary, and most “multi-agent pipelines” are really workflows that put an LLM in each stage.** That is often exactly right and nothing to apologize for: if the steps are knowable and fixed, the escalation principle says *write them down*, and a pipeline is you writing them down. A pipeline is frequently the correct, honest de-escalation from a flashier multi-agent design — fewer failure modes, trivially debuggable, cheap.

Where a pipeline crosses *back* over the boundary into genuinely agentic territory is when a stage gets the power to **decide the flow** — to loop back to an earlier stage on failure, to skip a stage conditionally, to choose the next stage from several. The moment a stage decides what happens next, you have a model in the control flow and you are back to pricing model-driven agency. A fixed chain is a workflow; a chain whose stages route is creeping toward a supervisor. Knowing which one you have built is the whole point of the Module 1 test.

12.6.2 What a Sequential Pipeline costs

Currency	What a Sequential Pipeline spends
Latency	Strictly additive and serial by definition — each stage waits for the one before. No parallelism to hide behind, but no controller round-trips either, so it is the lowest-overhead pattern here.
Tokens / money	One pass per stage, no decompose/aggregate/route calls on top. The cheapest centralized pattern, and often cheaper than a single agent solving the same task in a loop.
Failure modes	Error propagation — a bad output from an early stage poisons every stage after it, and a pure pipeline has no mechanism to catch or recover. Garbage in stage 1 is garbage out stage 3, confidently.

The characteristic failure is **error propagation down the chain**: stage 1 mis-extracts, and stages 2 and 3 dutifully score and write up the wrong data with no idea anything is amiss. Because the pipeline has no controller re-inspecting state between stages, nothing notices. The defenses are validation between stages (a schema check that fails loudly rather than passing garbage along) and, when recovery matters, giving a stage the power to reject-and-retry — which, note, is precisely the move that nudges the pipeline back across the boundary toward an agent.

12.7 The four patterns, in one frame

Centralized multi-agent design is, at bottom, choosing *where the control-flow decision lives* and *how often it is made*. That single pair of questions sorts all four:

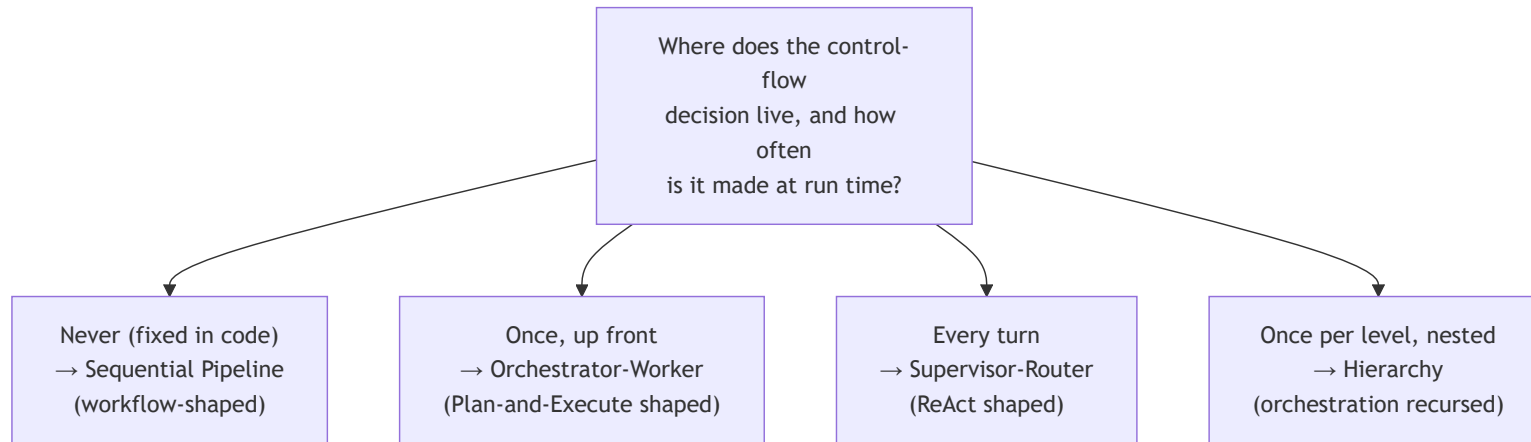


Figure 12.6: The four centralized patterns sorted by how often the controller decides the flow. A pipeline decides never at run time (you wired it). An orchestrator decides once, up front. A supervisor decides every turn. Hierarchy is orchestration nested inside orchestration. More decision points means more flexibility and more coordination tax.

The look-alikes that this module exists to separate — Orchestrator-Worker and Supervisor-Router — split cleanly on the middle two branches: **decide once and fan out** versus **decide every turn and route**. If you can say which of those a system does, you can name the pattern, and you will not call a turn-by-turn supervisor an orchestrator (or build an expensive per-turn supervisor when a one-shot decomposition would have done).

One closing reminder of the escalation principle, pointing the same direction it always points — *down*. Before any of these four, ask whether a single agent ([Module 8](#)) would have done the job; and before reaching for a supervisor or an orchestrator, ask whether a plain pipeline — a workflow — would have sufficed. Centralized is the cheap end of the multi-agent axis, and within it the pipeline is the cheap end again. The most agentic-looking option is, as ever, the one you must justify hardest.

12.8 Key Takeaways

- **Going multi-agent relocates complexity, it does not remove it** — you trade an overstuffed single context for a coordination problem, and every pattern here pays the *coordination tax*. The escalation principle governs the jump exactly as before: you earn a second agent by naming the specific way the single agent (Module 8) failed — context pressure, conflicting prompts, parallelism, independent verification, or domain-specific tooling — not by wanting one.
- **Centralized means one named component owns the control flow** — a controller decides who runs, in what order, and when the work is done; workers never decide among themselves. That single inspection point is what buys predictability and debuggability, which is why the escalation principle says start here before giving control away to peers (Module 10).
- **Orchestrator-Worker and Supervisor-Router look identical but differ in control flow** — the orchestrator decides the whole plan once, up front, and fans out (Plan-and-Execute at the system level); the supervisor decides one turn at a time and routes in series, re-deciding from the latest state (ReAct at the system level). Confusing the two is one of the most common multi-agent mistakes, and the Module 3 reasoning patterns reappearing one layer up is what makes the distinction concrete.
- **Each pattern inherits its characteristic failure and its cost from its shape** — Orchestrator-Worker fails by bad decomposition (and over-decomposition, a Module 12 anti-pattern); Supervisor-Router fails by misrouting and non-terminating ping-pong (the *No Termination Criteria* anti-pattern from Module 13, so every supervisor needs a hard cap); a Sequential Pipeline fails by error propagation down the chain because no controller re-inspects state between stages.
- **Hierarchy must follow the structure of the problem, not an org chart** — nest an orchestrator only where a sub-problem is genuinely large enough to decompose again. Layers that mirror how a human company would staff the task add pure coordination tax and are over-decomposition made structural; if you cannot name the sub-problem a layer decomposes, delete it.
- **The escalation principle keeps pointing down, even inside this module** — a pure pipeline is a *workflow* by the Module 1 test (your code owns the order), and most “multi-agent pipelines”

are honestly workflows with an LLM per stage. Before any of the four patterns ask whether a single agent would do; before a supervisor or orchestrator ask whether a plain pipeline would. Your mid-term “build a team” task decomposes cleanly into retrieve-candidates / score-fit / assemble — a textbook orchestrator-worker, and a useful test of whether you actually need one.

12.9 Connections and self-study

- **Module 1** — [Course Framing](#): the control-flow axis and the escalation principle this whole module rests on; the workflow/agent boundary the pipeline section revisits.
- **Module 3** — [Reasoning I — Reactive & Planning](#): Orchestrator-Worker is Plan-and-Execute and Supervisor-Router is ReAct, each lifted to the system level — the same termination requirement and cost shape come with them.
- **Module 8** — [Single-Agent Systems](#): the ceiling that justifies going multi-agent, and the de-escalation target you must rule out first.
- **Module 10** — [Decentralized Multi-Agent](#): what happens when you give the control-flow decision away from a single controller to peer agents.
- **Module 11** — [Hybrid Patterns](#): mixing centralized and decentralized control in one system.
- **Module 12** — [Architecture Anti-patterns](#): *over-decomposition* — the failure a needless hierarchy layer makes structural.
- **Module 13** — [Execution Anti-patterns](#): *No Termination Criteria*, which a supervisor’s per-turn routing loop inherits directly.
- **Primer** — [Multi-agent design patterns](#): the catalog these system-level patterns are drawn from, with the coordination-tax discussion in depth.
- **Primer** — [LangChain & LangGraph](#): how a supervisor-router is actually wired as a graph, and where the per-turn routing decision lives in code.
- **Project** — [Grant-Teams Mid-term](#): “build a team” decomposes into retrieve-candidates / score-fit / assemble — a natural orchestrator-worker, and a useful test of whether you actually need

one.

12.10 Self-check

You understand this module if you can answer these without notes:

1. What single property makes a multi-agent pattern *centralized*, and what does centralized control buy you at what cost?
2. Draw the control flow of Orchestrator-Worker and of Supervisor-Router, and state in one sentence each why they are *not* the same pattern despite looking alike.
3. Which Module 3 reasoning pattern does each of those two correspond to at the system level, and what does that correspondence tell you about their termination requirements?
4. State the rule for when a Hierarchy is justified, name the wrong instinct it guards against, and the Module 12 anti-pattern a needless layer produces.
5. Why does a pure Sequential Pipeline sit on the workflow/agent boundary, and what single change pushes it across into genuinely agentic territory?
6. Price each of the four patterns in the three currencies and name its characteristic failure mode.